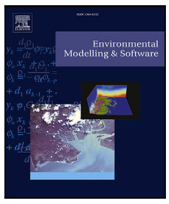
Environmental Modelling and Software 188 (2025) 106359



Contents lists available at ScienceDirect

Environmental Modelling and Software

journal homepage: www.elsevier.com/locate/envsoft





Efficient and fine-grained viewshed analysis in a three-dimensional urban complex environment

Yifan Zhang, Mengyu Ma *, Jun Li, Anran Yang, Qingren Jia, Zebang Liu

College of Electronic Science and Technology, National University of Defense Technology, Changsha 410073, China

ARTICLE INFO

Dataset link: https://github.com/Zoey-Fan/Q-view

Keywords:
Spatial viewshed analysis
Spatial visibility analysis
Urban oblique 3D model
Complex environment
Urban planning

ABSTRACT

Performing efficient and fine-grained viewshed analysis in 3D complex urban models, particularly when handling large-scale datasets, presents a significant challenge in Geographic Information Systems (GIS). Existing methods are primarily designed for 2.5D raster models and struggle to effectively manage large-scale data. Furthermore, the commonly utilized approaches for 3D models need large display memory and lack statistical analyses. To address these challenges, this paper adopted a results-oriented approach that diverged from the traditional data-driven paradigm by reformulating the conventional viewshed computation problem as a spatial query problem. Building on this premise, we proposed the Q-View method for oblique photogrammetry data, which involved creating an indexing model for large-scale datasets and enabled parallel querying between the line-of-sight (LOS) and the model. The Q-View method enables efficient and spatially exhaustive analysis, effectively overcoming the complexities associated with traditional viewshed computations. Experimental results showed that our approach achieved a query rate of up to 4 million visibility queries per second on a dataset with 17.6 million triangular meshes. Compared to the latest methods, it offered a 72.45% improvement in operational efficiency and superior accuracy relative to the GPU-rated method. These findings indicated that the proposed method substantially improved both the accuracy and efficiency of viewshed analysis in complex urban environments, providing decision support for urban planning and environmental monitoring.

1. Introduction

Spatial visibility analysis is based on the physiological functions of human vision, with the fundamental principle being the Line-of-Sight (LOS) propagation. The term "viewshed" is a critical concept in visibility analysis, referring to the area visible from a single or multiple viewpoints. These analyses are applied in a wide variety of fields, including site selection (Chen and Chen, 2021; Ma et al., 2023; Yao et al., 2023), route planning (Sanchez-Fernandez et al., 2021; Stucky, 1998), urban planning and monitoring (Helmrich et al., 2021; Gaube and Remesch, 2013) and traffic safety applications (Jung et al., 2018; Ma et al., 2019). Viewshed analysis is commonly performed on pseudo-3D (2.5D) and 3D scene models. In 2.5D models, each point has only one elevation value, regardless of the viewing direction, and these models typically represent simpler natural environments using Digital Elevation Models (DEM) or Digital Surface Models (DSM) (Zhang, 2021). In contrast, 3D models primarily consist of point cloud models and oblique photogrammetry models (hereafter referred to as oblique 3D models). Due to the voids inherent in point cloud models, which can lead to increased error rates in viewshed analysis, this paper focuses on oblique 3D models.

Oblique 3D models are used to spatially represent various complex features such as urban buildings, intersecting streets, and overpasses that present challenges not found in 2.5D models, as illustrated in Fig. 1. Oblique 3D models offer advantages such as high accuracy, refined model outputs and high fidelity. However, they typically comprise a large number of triangular meshes and extremely high-quality textures, resulting in enormous data volumes. For example, a city block model measuring 4 km by 4 km may contain 420 million triangular meshes. In the face of such a substantial volume of data, achieving efficient and highly accurate viewshed analysis poses a significant challenge. However, existing methods primarily focus on viewshed analysis in raster datasets, often involving a series of high-precision interpolations or geometric computations at the data level to achieve efficient and accurate results. The complexity of these computations can lead to reduced efficiency as the volume of data increases. Additionally, in complex urban environments, traditional analysis methods tend to produce significant errors due to the fact that the accuracy of 2.5D models diminishes. Currently, approaches for oblique 3D models include ray tracing, projection transformation, and the widely used GPU-rendered method. These methods typically project 3D models onto 2D planes or

* Corresponding author.
E-mail address: mamengyu10@nudt.edu.cn (M. Ma).

https://doi.org/10.1016/j.envsoft.2025.106359
Received 5 November 2024; Received in revised form 31 December 2024; Accepted 1 February 2025
Available online 6 March 2025
1364-8152/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

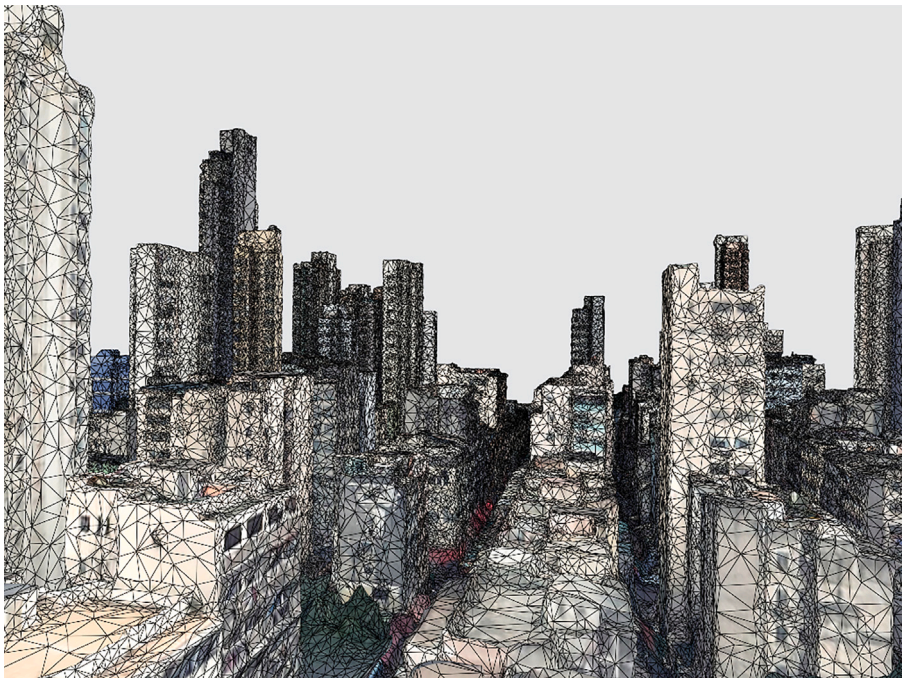


Fig. 1. Massive triangular meshes of building models.

generate shadows under lighting conditions to achieve rapid viewshed analysis. These algorithms and their improvements are widely used in many real-time 3D applications, enabling efficient viewshed analysis. Despite their excellent performance in real-time visualization, these methods still have limitations. GPU-rendered methods primarily display visualization results on the screen, but provide limited support for subsequent spatial computations, such as calculating cumulative visibility, visible point counts, and visibility indices. These limitations hinder the current viewshed analysis methods from playing a more significant role in decision-making evaluations across various application scenarios. Additionally, real-time rendering of viewshed in detailed scenes requires substantial display memory, leading to high operational costs. Therefore, it is crucial to develop and implement more refined 3D spatial analysis algorithms for high-precision viewshed analysis in complex scenarios.

To achieve efficient and accurate viewshed analysis on large-scale oblique 3D models, traditional methods typically require the identification of a specific data range for computation, followed by extensive calculations. In contrast, the approach proposed in this paper is results-oriented, enabling direct viewshed queries without the need to define a specific data range. (see Fig. 2). First, we organized and managed the large number of triangular meshes in the oblique 3D model, constructing a fine-grained query model that minimized the need for frequent computational access to the data. Next, we directly interacted with each LOS with the model, bypassing the projection or interpolation calculations typically required by conventional algorithms. This enabled rapid visibility determination for each LOS. Finally, we aggregated the results from all LOS to generate the overall viewshed map for the viewpoint.

The main contributions of this paper can be summarized as follows:

- We propose a novel paradigm for viewshed computation that transitions from the traditional, computation-intensive approach to a spatial query-based method. This approach directly returns sightline results, ensuring both efficiency and accuracy in viewshed analysis.
- We specifically introduce the Q-view method, which enables efficient and high-resolution viewshed analysis on large-scale datasets, offering significant applicability in urban planning and land use, particularly for site selection, placement of residential neighborhoods and urban roadway cameras.
- Our proposed model also supports visibility analysis for arbitrary curved sightlines, a feature not available in other algorithms.

The rest of this paper proceeds as follows. Section 2 reviews the research progress in spatial viewshed analysis. In Section 3, the techniques of Q-view algorithm are described in details. The Experimental results are presented and analyzed in Section 4 and the conclusions are drawn in Section 5.

2. An overview of previous work

2.1. 2.5D viewshed analysis

Most viewshed algorithms based on regular grids operate on 2.5D raster maps, such as DEM or DSM data. For instance, the classic viewshed algorithms such as R3, R2, and Xdraw (Blelloch, 1990; Shapira, 1990; Franklin et al., 1994; Ray, 1994) are based on line-of-sight calculations. The R3 algorithm is considered the most accurate, with a time complexity $O(n^3)$. In contrast, the R2 and Xdraw provide approximate viewsheds, reducing computational complexity at the expense of some accuracy. In subsequent studies on viewshed analysis, numerous methods have emerged to reduce computational costs by utilizing parallel computing and graphics processing units (GPUs) to enhance calculation efficiency. Axell and Fridén (2015) investigated the applicability of CPUs and GPUs for viewshed analysis, concluding that parallel GPUs are more suitable for raster-based viewshed analysis. Li et al. (2017) implemented the Franklin-Vogt algorithm in parallel using OpenMP and CUDA, achieving speedups of 17 times and 70 times compared to the serial algorithm respectively. Saxena and Kaur (2022) conducted a comparative analysis of the R2 and R3 viewshed methods on CPUs and GPUs, finding that the GPU-accelerated R2 algorithm had the lowest computational cost.

In addressing fine-grained scenarios and dense data, Zhao et al. (2013) tackled the challenges of large-scale real-time computation by employing a combination of coarse-scale and fine-scale spatial domain decomposition, a moving window for tiles, and scalable parallel processing. Their method achieved a performance that was 50 times faster than the R3 algorithm. Dungan et al. (2018) implemented a coarser viewpoint grid over the entire study area, using the center

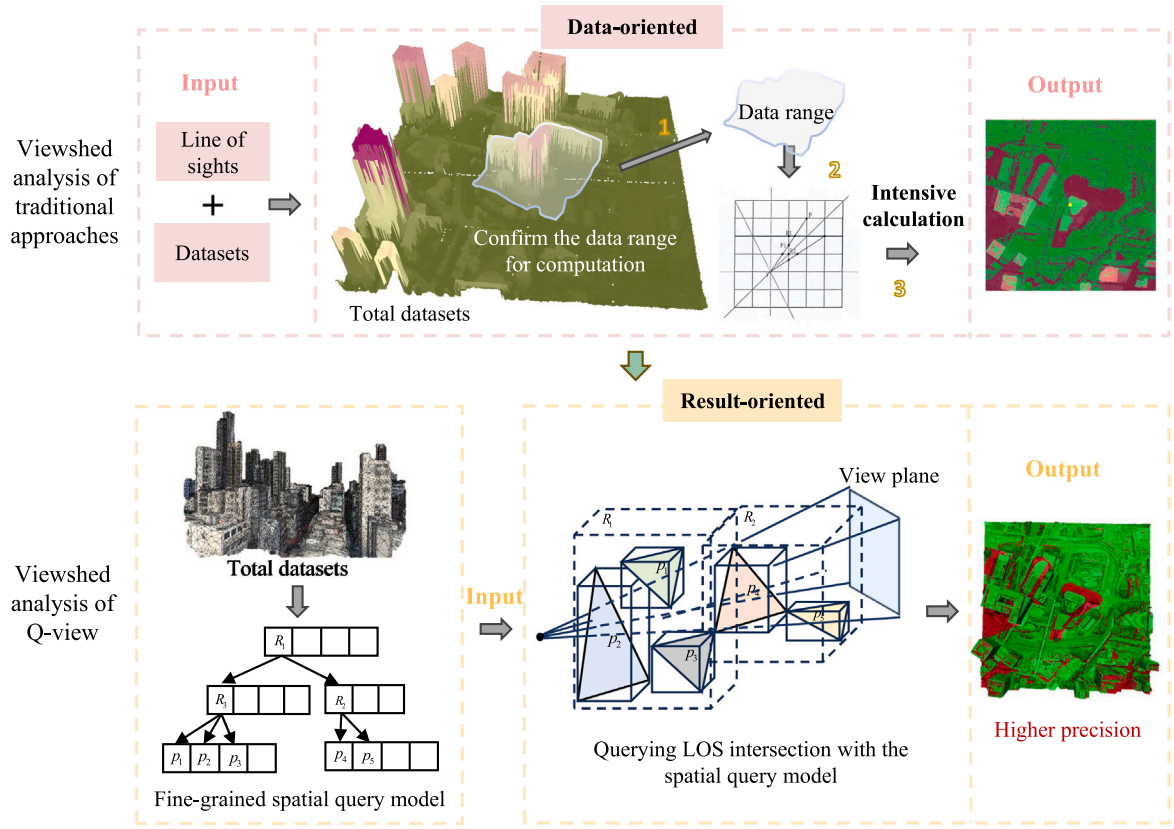


Fig. 2. The differences between Q-view and conventional approaches.

point of each grid cell in the DEM as the viewpoint. This approach effectively represented the overall visibility of the region while significantly reducing processing time. Wang and Dou (2020) developed a rapid candidate viewpoint filtering algorithm that considers viewshed overlap between viewpoints, specifically designed for multi-viewshed site planning. Wu et al. (2021) introduced the "Proximity-Direction-Elevation" (PDE) coordinate system and proposed the PDE Spatial Reference Line (PDERL) algorithm, enabling precise large-area viewshed analysis based on DEMs. Wang et al. (2023) proposed a view-tree method to efficiently compute viewsheds from DEMs by constructing a view-tree structure that extracts LOS between observers and targets, enhancing both computational efficiency and accuracy. Viewshed analysis algorithms based on regular DEM raster maps primarily focus on terrain data with single-type features, rendering them unsuitable for complex urban 3D scenarios.

2.2. 3D viewshed analysis

Feng et al. (2015) proposed a real-time viewshed analysis algorithm for digital earth applications based on GPU parallel computing. This algorithm creates occlusion bodies to mask geometric features near the viewpoint, with occluded areas marked by the stencil buffer in a programmable 3D visualization pipeline. Anderson and Rex (2019) analyzed drivers' viewsheds by integrating digital terrain models (DTMs), radar point cloud data, and distance decay weighting to assess vegetation visibility while driving. Chen and Chen (2021) proposed a method based on a field-of-view (FOV) model to delimit spatial boundaries and utilized GPU-based 3D scene depth computation for viewshed visualization. This approach enables multi-point viewshed analysis in complex urban environments. Orlof and Ozimek (2021)'s method, which uses radial triangulated irregular networks(TIN) surfaces and multiple computers, subdivides viewshed visualization tasks to expedite the generation of comprehensive viewshed. Labib et al. (2021) employed multi-source spatial data and distance decay weighting to model FOV, achieving high-resolution viewshed analysis results. To address the increasing demand for high-resolution data computation and acceleration, Jung et al. (2018) arranged entire point cloud data on the x-y plane to generate a grid representation of roads and surrounding objects. They performed sightline analysis on the grid to efficiently detect target points in the driver's field of view and then evaluated these points in 3D space to determine visibility obstructions, resulting in the creation of driver visibility maps. Parent and Lei-Parent (2023) designed a viewshed analysis method based on the spatial relationship between shortest paths and sightlines, suitable for large-scale viewshed analysis of buildings and vegetation. Wróżyński et al. (2024) combined low-resolution terrain models with high-resolution vegetation and building models, using Blender software to calculate viewshed over large areas. Despite the potential of these extensive triangular mesh models to enhance GPU visualization rendering, their spatial analysis capabilities remain limited, hindering in-depth data mining and analysis.

2.3. Comparing viewshed and simulation

Spatial query techniques are increasingly employed in simulation fields such as collision detection in games or animations. Wang and Cao (2021) reviewed various methods for collision detection of deformable objects, including bounding volume hierarchies, spatial partitioning, sweep and prune algorithms, and self-collision detection techniques. While these concepts share similarities with the approach proposed in this paper, their core objectives differ significantly. Collision detection is a complex process primarily focused on determining interactions between two or more models to assess whether a collision occurs, and in more complex scenarios, it must account for object deformations. The research focus is thus centered on dynamic and efficient query techniques (Shi et al., 2023; Sabino et al., 2023). In contrast, viewshed

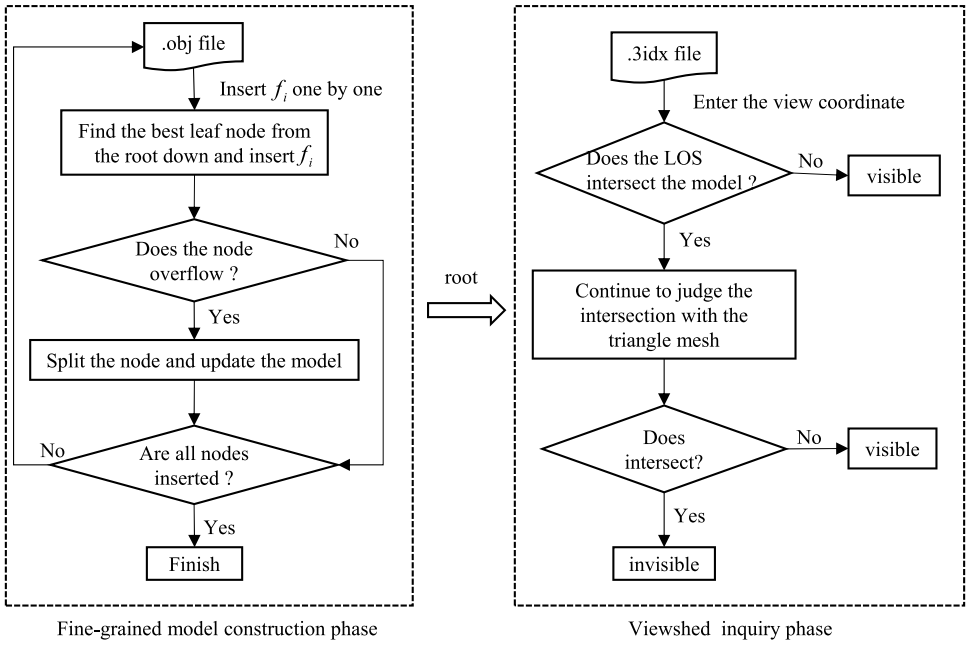


Fig. 3. The flowchart of Q-view algorithm.

analysis emphasizes how to quickly determine visibility and occlusion relationships in static scenes. Spatial query techniques hold significant appeal for large-scale data viewshed analysis, making them particularly valuable in this context.

3. Methodology

The Fig. 3 illustrates the overall process of the Q-view algorithm, which consists of two phases: index construction and spatial intersection querying. In the index construction phase, all triangular meshes are organized to construct the fine-grained index model and the algorithm then advances to the LOS spatial intersection query phase. During this phase, each line of sight is subjected to two query evaluations, yielding a boolean value as the result.

3.1. Fined-grained index model construction

In recent years, advances in computer graphics and computational technology have significantly enhanced 3D visualization and spatial analysis across various fields. Among the modeling approaches supported by Industry Foundation Classes (IFC), triangular mesh models are the most prevalent. This method represents real-world objects using a series of triangular meshes and texture maps. Typically, these models store coordinate points directly, forming triangles from 3D spatial coordinates, and encompass these triangles to create 3D spatial objects. Triangular mesh models offer several key features:

1. Simple Data Structure: They facilitate fast rendering and lighting calculations, and are highly suitable for parallel processing.

2. Extensive Data Acquisition: The use of 3D scanning technology allows for the generation of a large number of triangular meshes.

3. Precision and Accuracy: With a sufficient number of meshes, they can approximate complex surfaces with high precision.

Triangular mesh models are commonly stored in the OBJ file format, which supports curves, surfaces, point groups, polygons with three or more points, as well as normal and texture coordinates. The obj file format defines spatial objects in a textual form, with each line representing a spatial object composed of keywords and associated data. The data structure of triangular mesh models is divided into vertex and face information. Given the immense number of meshes in these models, which can reach billions within an area of 10 square

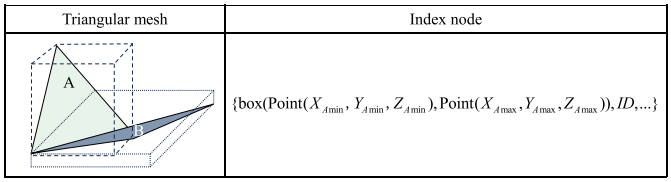


Fig. 4. A three-dimensional fine-grained spatial index model structure.

kilometers, performing viewshed calculations at such a scale incurs significant computational overhead, resulting in low efficiency. To address this issue, it is essential to pre-organize and manage data of such a large scale to minimize frequent access to irrelevant data. In this paper, we utilize 3D R-tree for modeling, enabling efficient querying. The 3D R-tree, introduced by Guttman (1984), efficiently organizes and retrieves three-dimensional spatial data. In a 3D R-tree, leaf nodes contain specific data objects, and spatial indexing begins at the root node. The search process narrows down through the tree based on numerical ranges until the leaf nodes, significantly enhancing retrieval efficiency. The 3D R-tree employs Axis-Aligned Bounding Boxes (AABB) to enclose data items. Starting from the leaf nodes, AABBs encase triangular mesh data as illustrated in Fig. 4. Parent nodes then use AABBs to encompass their child nodes' AABBs. As the hierarchy ascends, the AABB expands to cover larger spatial areas, culminating in the root node's AABB enclosing the entire dataset. To construct the fine-grained spatial index, triangular mesh data are inserted sequentially. Nodes are dynamically split based on a set threshold, adjusting the tree's height accordingly. Algorithm 1 outlines the construction process of this fine-grained model.

To illustrate the operations more intuitively, Fig. 5 explains the process of inserting a triangular mesh into an index model. The core idea is to construct an AABB for each triangular mesh and build the 3D R-tree from the bottom up. $p_1, p_2, \ldots, p_5$ represent the spatial triangular meshes of the original data, with solid line cubes representing their AABBs, defined by the minimum $(X_{\min}, Y_{\min}, Z_{\min})$ and maximum $(X_{\max}, Y_{\max}, Z_{\max})$ coordinate points. Following the principles of R-trees, the integration process continues upward to form the index space R_1, R_2 corresponding to the root node. With the constructed

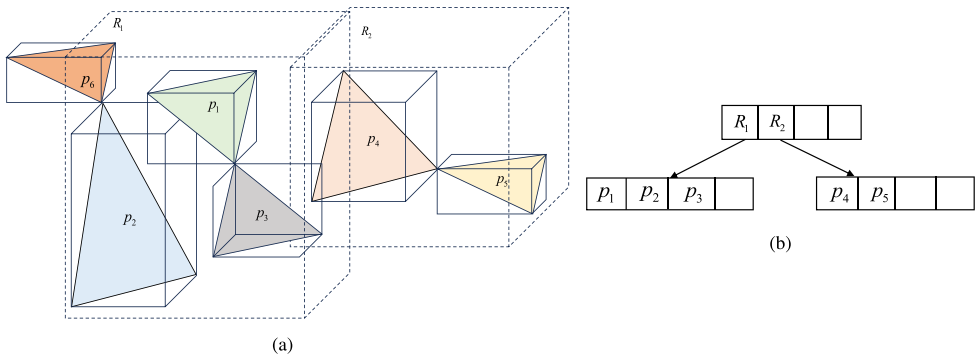


Fig. 5. Based on index model and AABB for triangular meshes.

Algorithm 1 Constructing a fine-grained index model.

Input: triangular meshes

1. Obtain the set of triangular meshes F, where each triangular mesh is denoted as f;
2. Find the node N with minimum AABB expansion;

if not `exceedsThreshold`(N, f) **then**
3. $N \leftarrow$ Node;
else
4. $(N, N1) \leftarrow$ `splitNode`(N, f); // Split the node into N and $N1$.
end if
while N is not root **do**
5. $L \leftarrow$ `findParentNode`(root, N); //N is not the root, adjust the parent node L.
6. `adjustAABB`(L, N, N1);
if `exceedsMaxStorageConstraint`(L) **then**
7. $(L, L1) \leftarrow$ `splitNode`(L, N1); //if L exceeds the $max_threshold$, then split L.
end if
8. $N \leftarrow L$;
9. $N1 \leftarrow L1$;
end while
return root

Output: fine-grained spatial index model

spatial index database, the AABBs of each triangular mesh can be quickly queried, facilitating the rapid retrieval of spatial index data that intersects with LOS. When inserting a new triangular mesh p_6, the minimal expansion node is queried first. The root node contains two AABBs, R_1, R_2. A comparison reveals that R_1 partially encloses p_6, while inserting p_6 into R_2 would significantly increase its range. Therefore, R_1 is chosen as the insertion node. However, since the maximum number of objects that a node can store in this tree is limited to 3, a split operation is required for the insertion node.

The root node R_1 is split into two nodes R_1, R_1'. Based on the nearest neighbor strategy p_2, p_6 are placed into one node, while p_1, p_3 are placed into the other node. This change is then propagated upward. Since the parent node contains only R_1, R_2 and does not exceed the threshold, further splitting is unnecessary. Instead, the range is adjusted to ensure the leaf nodes are properly enclosed. The resulting structure of the 3D R-tree after the split is shown in Fig. 6.

3.2. Viewshed query with the fined-grained index model

For visibility analysis between two points, a spatial intersection query is conducted between the LOS and the indexing model. If an intersection occurs, the two points are not mutually visible; otherwise, the target point is considered visible. This problem can be formulated as a geometric challenge that involves the spatial intersection of LOS with the AABB, followed by the intersection of the line segment with a triangle.

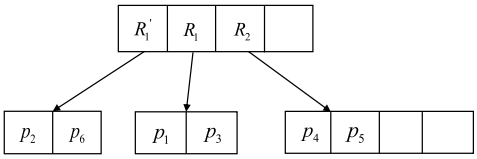


Fig. 6. Node insertion diagram.

The spatial intersection query algorithm between LOS and the AABB uses the Slab method from Preparata and Shamos (2012). As shown in Fig. 7, let the LOS be $\overrightarrow{P}_{start}, \overrightarrow{P}_{end}$, respectively, and the maximum and minimum point of the AABB are $A(X_{\max}, Y_{\max}, Z_{\max})$ and $B(X_{\min}, Y_{\min}, Z_{\min})$, respectively. Any point along the LOS can be represented by Eq. (1).

$$\overrightarrow{P} = \overrightarrow{P}_{start} + t * \overrightarrow{D}, (0 \leqslant t \leqslant 1) \tag{1}$$

where $\overrightarrow{P} = \overrightarrow{P}_{end} - \overrightarrow{P}_{start}$ represents the direction of the LOS, and t indicates the proportion of the distance from the starting point to any point along the LOS. The core principle is that the intersection of the LOS with the AABB is determined by its intersection with the AABB's six faces. This process can be simplified to analyzing the relationship between the maximum entry distance, $max(t_{x_near}, t_{y_near}, t_{z_near})$, and the minimum exit distance, $min(t_{x_far}, t_{y_far}, t_{z_far})$. If the maximum entry distance is less than the minimum exit distance, then the LOS intersects the AABB. For instance, the intersection with the X-slab can be determined using Eq. (2).

$$x_{start} + t * d_x = X_{min} \tag{2}$$

Similarly, the coefficients t for the Y-slab and Z-slab can be calculated. The calculation pseudocode is shown in Algorithm 2.

To enhance the precision of the judgment results, this paper further evaluates the spatial relationship between the LOS and the mesh. Möller and Trumbore (2005) efficiently calculates the intersection point between the LOS and a triangular mesh in 3D using vector and matrix operations, thereby avoiding the need to pre-compute the triangle's plane equation. For a triangle defined by vertices P_0, P_1, P_2, the intersection with the LOS can be determined as Eq. (3).

$$\overrightarrow{P}_{start} + t * \overrightarrow{D} = (1 - v - u) * \overrightarrow{P}_0 + u\overrightarrow{P}_1 + v\overrightarrow{P}_2 \tag{3}$$

Here, $(u, v) \in \Omega = \{(u, v) : u \geq 0, v \geq 0, u + v \leq 1\}$ are the barycentric coordinates. By solving the constraints on the variables t, u, v, we can ascertain whether an intersection occurs. The above Eq. (4) can be

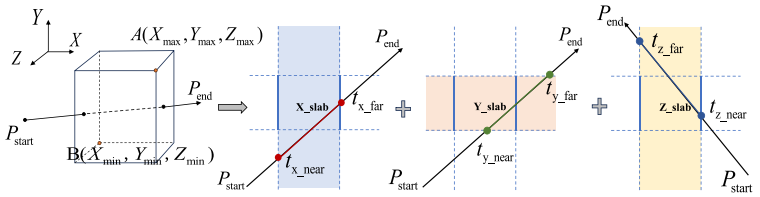


Fig. 7. The intersection of the LOS and the AABB.

Algorithm 2 Intersection query check between the LOS and the AABB

Input: Maximum and minimum values of AABB, start and end point coordinates.

for each coordinate axis **do**
  if line segment is parallel to the axis && start point is on both sides of the axis **then**
    return False
  else
    Calculate the diagonal distance from the line segment to the plane.
    if intersects **then**
      return True
    else
      return False
    end if
  end if
end for
Output: Judgment Results

Algorithm 3 The intersection check between the LOS and the triangular mesh

Input: Coordinates of the three vertices of the triangle, start and end point coordinates of the LOS

1. Calculate $\vec{E}_1, \vec{E}_2, \vec{S}, \vec{S}_1, \vec{S}_2$

if $\vec{S}_1 \cdot \vec{E}_1 == 0$ **then**
  return False
else
  if $u \geq 0 \parallel v \geq 0 \parallel u + v \leq 1 \parallel 0 \leq t \leq 1$ **then**
    return True
  end if
  return False
end if

Output: whether the intersection point is inside the triangular mesh.

solved using Cramer's rule:

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\vec{S}_1 \cdot \vec{E}_1} \begin{bmatrix} \vec{S}_2 \cdot \vec{E}_2 \\ \vec{S}_1 \cdot \vec{S} \\ \vec{S}_2 \cdot \vec{D} \end{bmatrix} \tag{4}$$

Where the vector parameters are defined as Eq. (5):

$$\begin{cases} \vec{E}_1 = \vec{P}_1 - \vec{P}_0 \\ \vec{E}_2 = \vec{P}_2 - \vec{P}_0 \\ \vec{S} = \vec{O} - \vec{P}_0 \\ \vec{S}_1 = \vec{D} \times \vec{E}_2 \\ \vec{S}_2 = \vec{S} \times \vec{E}_1 \end{cases} \tag{5}$$

If the obtained solution satisfies $(u, v) \in \Omega$, and $t \in [0, 1]$, then the intersection point of the line and the plane lies within the triangle; otherwise, it lies outside the triangle. If $u = 0, v = 0$, or $u + v = 1$, the intersection point is on the edge of the triangle. Algorithm 3 outlines the process for determining the intersection between the LOS and the triangle. For identified triangular meshes, in addition to assessing their visibility, semantic recognition can also be performed to support intelligent decision-making. This topic will be explored in future work and is not discussed in detail here.

In visibility analysis, the LOS is a connected path between an observer and a target object. Traditionally, this concept implies a straight line. However, in contemporary visibility concepts, it can also encompass trajectories described by mathematical methods, including straight lines, parabolas, continuous polylines, and other shapes. The core idea of Q-view algorithm is to construct a fine-grained spatial model for triangular meshes and perform viewshed analysis by determining the intersection of the LOS with the model. This algorithm is suitable not only for visibility analysis of straight segments between two points but also for curves. For example, when considering a parabola as shown in Fig. 8, the algorithm first discretizes the parabola into several straight segments that approximate the parabolic curve. Each discrete straight segment is then subjected to the proposed algorithm for visibility analysis. By sequentially applying the visibility analysis to each segment, the visibility of the entire curve can be assessed. If an intersection occurs during the query process, the remaining trajectory of the curve is deemed non-visible.

3.3. Parallel processing architecture

This paper decomposes viewshed into visibility analysis between multiple points. Since each LOS interacts independently with the environment, this approach is highly suitable for parallel programming models. OpenMP (Open Multi-Processing) is an application programming API used for programming in C, C++, and Fortran on shared address space computers. The OpenMP execution model follows a fork-join execution pattern, as illustrated in Fig. 9. The program preloads the spatial index during the main thread. Upon entering the parallel domain, the main thread creates a fork, which then executes the parallel region code concurrently with the main thread. During each parallel phase, the program generates random numbers to represent observation points and target points. The main for-loop for viewshed analysis initiates parallel computation, where each LOS independently interacts with the spatial index to obtain boolean values indicating the visibility of each target point. After executing the parallel region code, only the main thread continues to run the serial code (join). Finally, the program outputs the time taken for the parallel computation, which is used to analyze the parallel speedup of the Q-view algorithm. Algorithm 4 illustrates the code executed in parallel.

4. Results

In this section, we conducted extensive experiments to evaluate the performance of Q-view algorithm. First, we performed both qualitative and quantitative comparison experiments in 2.5D and 3D environments. We compared our algorithm with the classical viewshed algorithms R2 and R3, as well as view-tree proposed by Wang et al. (2023) in the 2.5D environment. In the 3D environment, we compared Q-view with the GPU-rendering method, specifically evaluating the viewshed

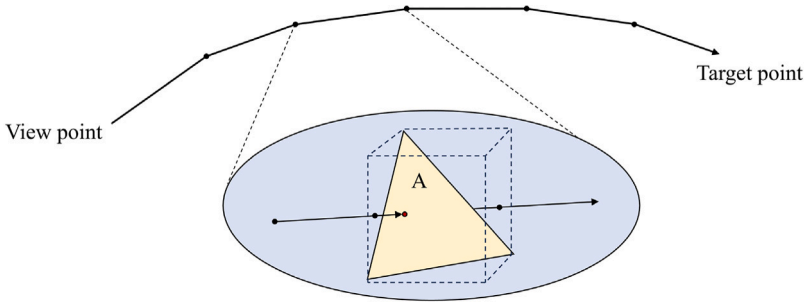


Fig. 8. Visibility analysis of curve.

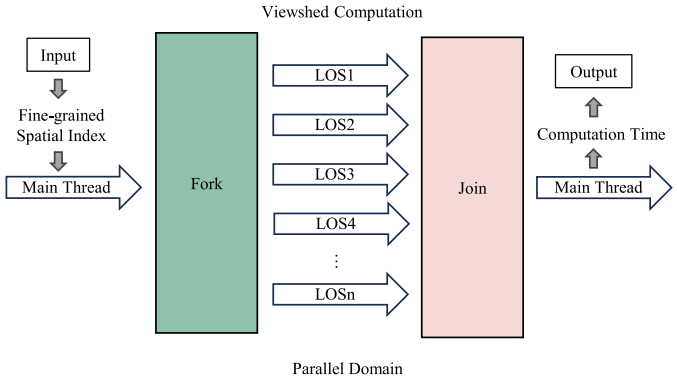


Fig. 9. Fork-Join parallel execution model.

Algorithm 4 Parallel viewshed analysis.

Input: spatial index model

```
#pragma omp parallel {
   Generate random numbers independently
   #pragma omp single { Record start time }
   #pragma omp for schedule(dynamic) { Perform visibility analysis
}
   #pragma omp single { Record end time } }
```

Output: computation time

Table 1
Environment setup.

Project	Description
CPU	32 cores,Intel(R) Xeon(R) E5-2620@ 2.10GHz
Memory	512G
Operating System	Ubuntu 20.04

analysis module of the CesiumLab software. Our primary focus is on analyzing the accuracy and efficiency of viewshed analysis for large-scale datasets using triangular meshes. Additionally, we assessed the performance of Q-view in terms of visibility as the dataset size increases. Furthermore, we examined the parallel computing performance of Q-view, comparing its efficiency to serial computation as the number of threads increases. Finally, We explored the feasibility of curve visibility analysis in an extended experiment. The experimental code is written in C++, and the detailed experimental setup is shown in Table 1.

Table 2 presents the datasets used for the experiments, which can be downloaded from the CSDI website. These datasets consist of oblique 3D models of varying scales, detailing the number of triangular meshes and the data range for each model.

Table 2
Experiment datasets of varying scales.

Dataset	Triangular mesh(million)	Data range(meter)	Minimum point
Dataset-1	0.87	250 × 250	(833250,812500)
Dataset-2	4.07	500 × 500	(833250,812500)
Dataset-3	17.61	1000 × 1000	(833000,812250)
Dataset-4	75.10	2000 × 2000	(832750,811750)
Dataset-5	416.85	4000 × 4000	(832750,811500)
Dataset-6	804.43	10240 × 7380	(833245,816360)
Dataset-7	1603.19	14500 × 8500	(830000,809000)
Dataset-8	54.52	2000 × 2000	(834872,818398)
Dataset-9	6.78	500 × 500	(834872,818398)

4.1. Accuracy comparison between different methods

We selected Dataset-9, a 500 × 500 meter area near Victoria Harbour, close to the Hong Kong Special Administrative Region Government Headquarters. Fig. 10(a) displays the oblique 3D model of Hong Kong, consisting of 6,776,701 triangular meshes. Fig. 10(b) shows the DSM model converted from the oblique 3D model, which is used as the input data for R2, R3 and view-tree.

We set the same observation position at 22.2806° N, 114.1670° E, at a height of 100 m. For each algorithm, we varied the observation radius to perform viewshed analysis within a square area centered on this position. The results of the four algorithms are shown in Fig. 11. The qualitative results indicated that the viewshed analysis outcomes of the R2 and R3 algorithms are nearly identical. However, view-tree algorithm showed significant differences, primarily because each node can only acquire information from its parent node. This could cause calculation errors to accumulate, leading to discrepancies. Q-view algorithm's result also showed significant differences in the invisible areas near the viewpoint compared to the R2 and R3 algorithms. The following section will provide a detailed analysis of these differences.

Upon closer examination of the buildings near the viewpoint, the red line frames in Fig. 12 highlight areas where building tops appear jagged after conversion. This results in a sawtooth pattern in the invisible areas near view-point for R2 and R3. This effect is due to elevation extraction and interpolation errors during the data conversion process, especially in complex building structures, which affect the accuracy of traditional algorithms. In contrast, the proposed method leverages the high precision of the oblique 3D model to more accurately preserve and process the fine structures of buildings. Specifically, our algorithm's detailed handling of triangular meshes when constructing the spatial index allows for a smoother representation of non-visible areas near the viewpoint obstructed by tall buildings. This smooth continuity reduces the sawtooth effect and more accurately reflects the actual terrain and building features. As a result, our algorithm demonstrates higher accuracy and reliability in the invisible areas near the viewpoint, significantly outperforming the traditional R3 algorithm and view-tree in terms of accuracy. This improvement provides more reliable outcomes in large-scale and high-complexity application contexts.

Based on the above comparisons, Q-View demonstrates higher accuracy and reliability than the most precise traditional R3 algorithm. To



(a) (b)

Fig. 10. Data models. (a) The oblique photographic model. (b) The DSM model.

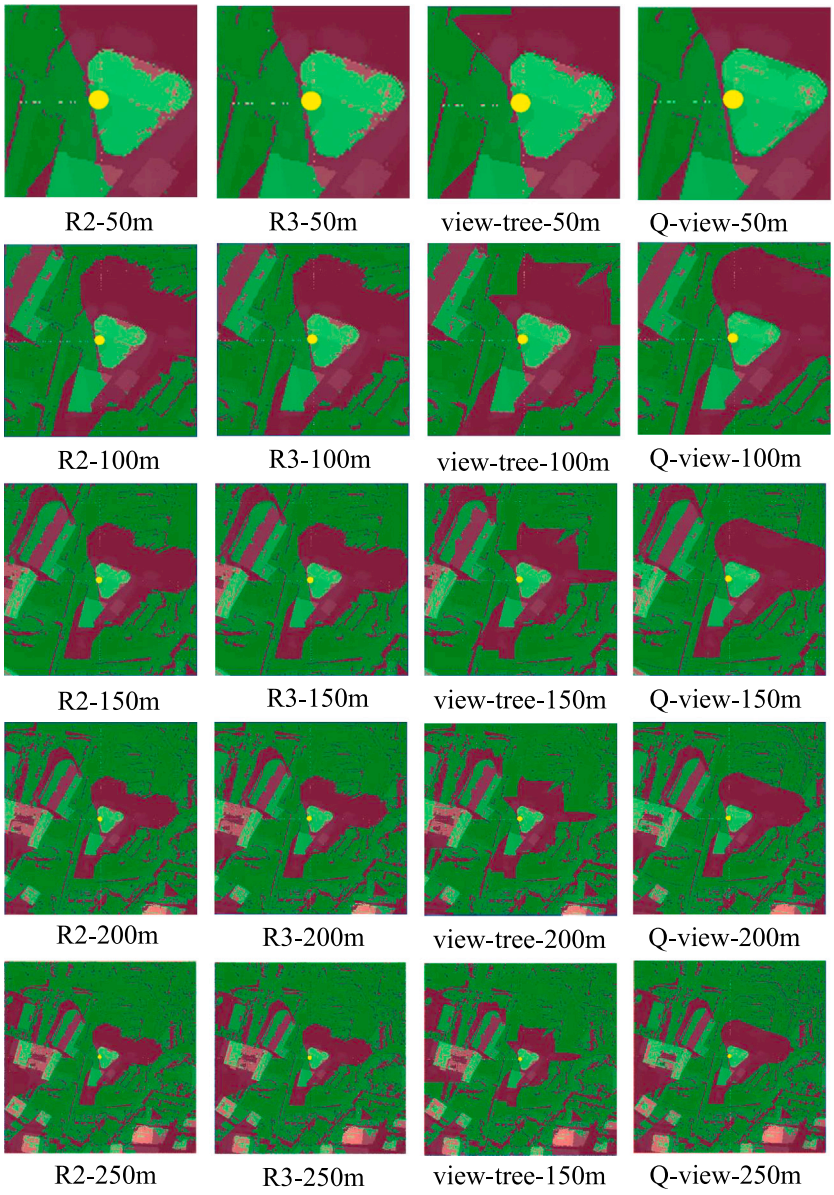
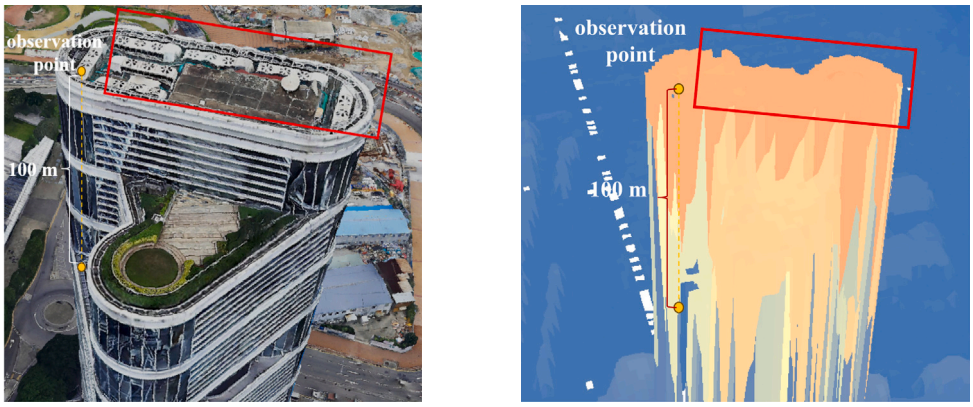


Fig. 11. Viewshed results for four algorithms at different radii.



(a) (b)

Fig. 12. Data transformation differences near the observation point.

Table 3
Error rate and omission rate of each algorithm with different radius.

	view-tree		R2		R3	
	η_{er} (%)	η_{or} (%)	η_{er} (%)	η_{or} (%)	η_{er} (%)	η_{or} (%)
50 m	24.3	23.1	26.9	22.9	27.1	25.1
100 m	43.5	33.6	45.3	31.3	45.5	31.1
150 m	56.2	27.7	55.8	27.7	55.9	27.6
200 m	62.7	28.5	61.9	29.0	62.1	28.9
250 m	61.9	31.5	61.1	32.4	61.2	32.3

further quantify the gap between traditional algorithms and Q-View, we used the error rate and omission rate as the main evaluation metrics, following (Wang et al., 2023):

$$\eta_{er}(X) = \frac{N_{Los_Rtree=invisibile} \cap N_{X=visible}}{N_{Los_Rtree=invisibile}} \tag{6}$$

$$\eta_{or}(X) = \frac{N_{Los_Rtree=visibile} \cap N_{X=invisible}}{N_{Los_Rtree=visibile}} \tag{7}$$

The error rate $\eta_{er}(X)$ refers to the ratio of areas that are not visible in Q-view but are misjudged as visible. The omission rate $\eta_{or}(X)$ refers to the ratio of areas that are visible in Q-view but are misjudged as invisible. These two metrics are used to measure the accuracy gap between DSM-based algorithms and Q-view based on a 3D model. The data in Table 3 indicate that R2 and R3 produce similar overall results. View-tree shows a significant increase in error rate, indicating a higher probability of misclassification. Additionally, the error and omission rates for all three algorithms increased with the radius, suggesting poor performance in large-scale applications. This trend highlights the limitations traditional raster-based algorithms for viewshed analysis in complex scenarios. In applications requiring precise 3D spatial viewshed analysis, these algorithms fail to meet practical demands.

To further demonstrate the effectiveness of Q-view on oblique 3D models, we qualitatively compared Q-view with the GPU-rendered method. The GPU-rendered method is a fast algorithm for generating shadows in a 3D scene, rendering parts not visible to the light source as shadows. Its rendering speed is independent of scene complexity and data type. However, the GPU-rendered method suffers from issues such as shadow acne and aliasing, which reduce the reliability of viewshed analysis results.

Furthermore, although the GPU-rendered method can achieve real-time viewshed rendering at different resolutions, its accuracy depends largely on the depth map's resolution. Insufficient display memory may cause the system to prioritize caching essential on-screen data, neglecting off-screen data. This can lead to the loss of depth information for off-screen objects in dynamic scenes, affecting shadow calculations. Fig. 13 illustrates the viewshed effects at 20-meter and 10-meter resolutions at the same view point. While the 10-meter resolution provides more detailed features, significant viewshed analysis errors still occur. This comparison highlights the limitations of the GPU-rendered method in maintaining accurate visibility results, particularly in complex scenes with high detail requirements. In contrast, Q-view provides more reliable and precise visibility analysis by leveraging the high-resolution features of oblique 3D model, demonstrating its superiority in detailed 3D spatial analysis.

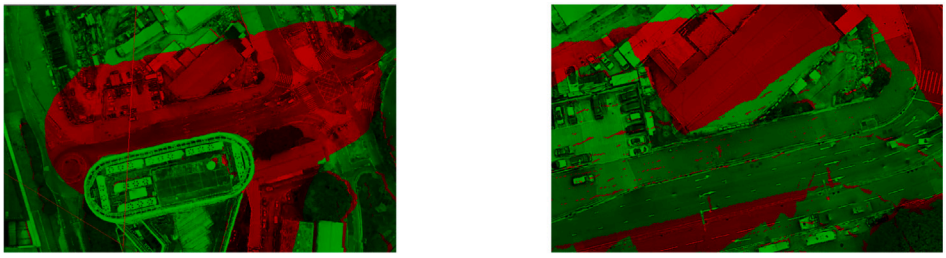
The viewshed area generated by Q-view was overlaid onto the oblique 3D model to generate the viewshed effect, as shown in Fig. 14(a). The viewshed effect using the GPU-rendered method is shown in Fig. 14(b). At a height of 150 m, the viewshed effect shows similar results to Q-view and GPU-rendered method in overall analysis. This indicates that the proposed algorithm effectively utilizes the high-precision data of oblique 3D models for more detailed viewshed analysis of complex 3D structures. Although the GPU-rendered method is widely used in 3D viewshed analysis due to its real-time rendering advantages, its accuracy is constrained by GPU display memory limitations. Consequently, it cannot achieve the same level of analysis depth as Q-view. Therefore, Q-view holds greater potential for high-precision 3D viewshed analysis, especially in complex scenarios that require fine-grained analysis, where its advantages are more pronounced.

Fig. 15 effectively illustrates the results of the 3D analysis, with observation points located on the roof (114.16594°, 22.28059°, 2 m), ground (114.16813°, 22.28118°, 2 m) and facade (114.16668°, 22.27933°, 0.1 m), each at a radius of 50 m. The observation direction for the wall is defined as (1, 1, -1).

4.2. Efficiency comparison between different methods

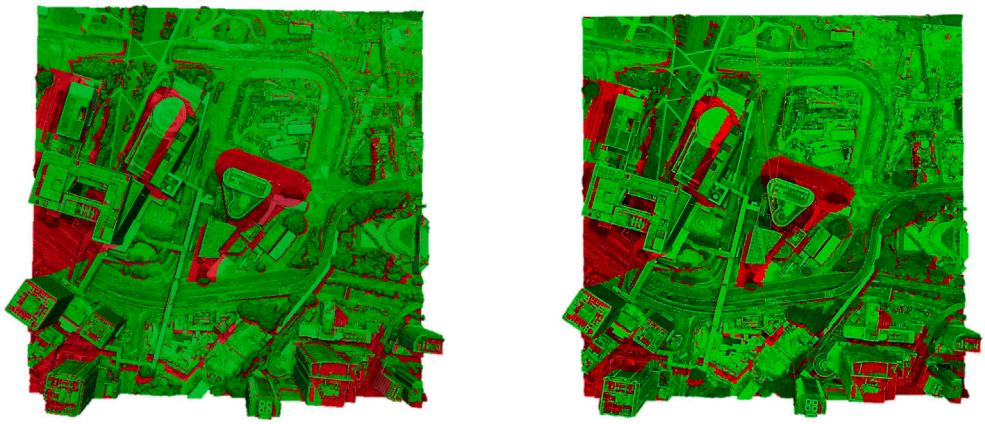
We compared the proposed algorithm with R2, R3, and view-tree on Dataset-8, containing 54.52 million meshes. The observation points were set at the same locations and heights. Due to long runtimes and crashes of the other three algorithms on ultra-large datasets, we controlled the data volume by varying the observation radius within a single dataset. The observation radii were set sequentially to 200, 400, 600, 800, and 1000 m. This setup enabled us to test how the runtime of each algorithm varied with increasing observation radius. The results are shown in Table 4.

Q-view and view-tree are substantially more efficient than R2 and R3. Despite its highest precision, R3 exhibits relatively low efficiency with a complexity of $O(n^3)$. In the same dataset, with a constant query time, Q-view has a time complexity of approximately $O(n^2)$, similar to that of R2 and view-tree. The absolute times of the other three CPU-based methods may vary due to different operating environments, leading to significant time differences between algorithms. We use



(a) (b)

Fig. 13. Viewshed of the GPU-rendered method at different resolutions. (a) 20-meter resolution. (b) 10-meter resolution.



(a) (b)

Fig. 14. Viewshed results of 3D methods. (a) Viewshed of the Q-view. (b) Viewshed of the GPU-based method.

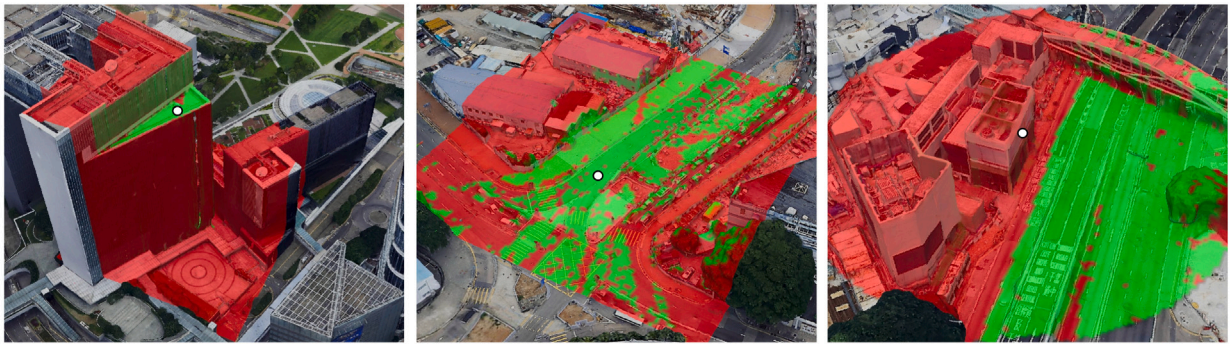


Fig. 15. The 3D viewshed effects of the observation points are sequentially on the roof, the ground, and the facade.

Table 4
The efficiency comparison of four algorithms.

	View-Tree	R2	R3	Q-view
200 m	0.62 s	7.80 s	384.51 s	0.19 s
400 m	2.42 s	30.89 s	3039.24 s	0.68 s
600 m	5.66 s	70.10 s	10513.40 s	1.5 s
800 m	9.82 s	126.48 s	25190.00 s	2.54 s
1000 m	15.73 s	202.02 s	50545.60 s	4.14 s

relative time to characterize the runtimes of Q-view, R2, and view-tree relative to R3. Let $\delta_t(X_i)$ represent the time ratio of algorithm X_i:

$$\delta_t(X_i) = \frac{T_{X_i}}{T_{R_3}} \tag{8}$$

Q-view has the lowest time ratio, indicating the highest relative efficiency, as shown in Fig. 16. On average, the runtime efficiency of Q-view is 72.45% higher than that of view-tree and 97.83% higher than that of R2. Additionally, although Q-view analysis takes longer compared to the GPU-rendered method, it supports a larger-scale data analysis, and the computation time is acceptable. To further investigate the performance of the four algorithms under different data scales, we analyzed the standard deviation of the runtime for each algorithm. Q-view has the lowest standard deviation at 1.42, while the standard deviations for R2, R3, and View-Tree are 49, 13063, and 2.85 times higher, respectively. The results here and in Section 4.1 indicate Q-view achieves higher accuracy than R3 in less time. Although view-tree also exhibits high efficiency, it based on a large sacrifice of precision. This efficiency makes Q-view a viable option for extensive and fine-grained viewshed analysis within an acceptable time frame.

Table 5
The average visibility query performance and average runtime on different datasets.

Dataset	Visibility query Performance (million operations per s)	Mean time (s)
Dataset1	3.79	0.016
Dataset2	3.96	0.063
Dataset3	4.01	0.25
Dataset4	3.65	1.10
Dataset5	3.56	4.49
Dataset6	2.26	33.38
Dataset7	2.46	50.09

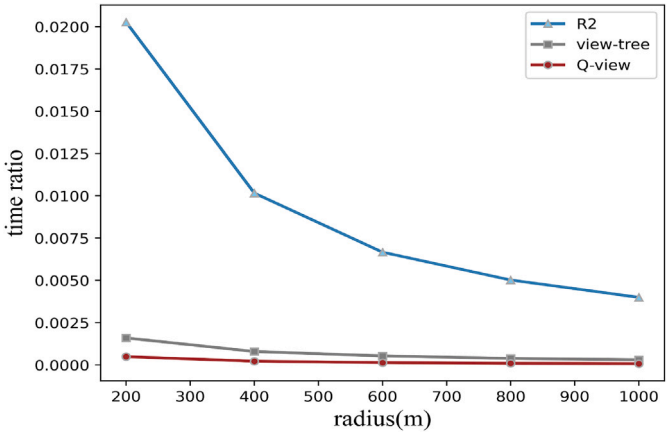


Fig. 16. The time ratio based on R3 algorithm.

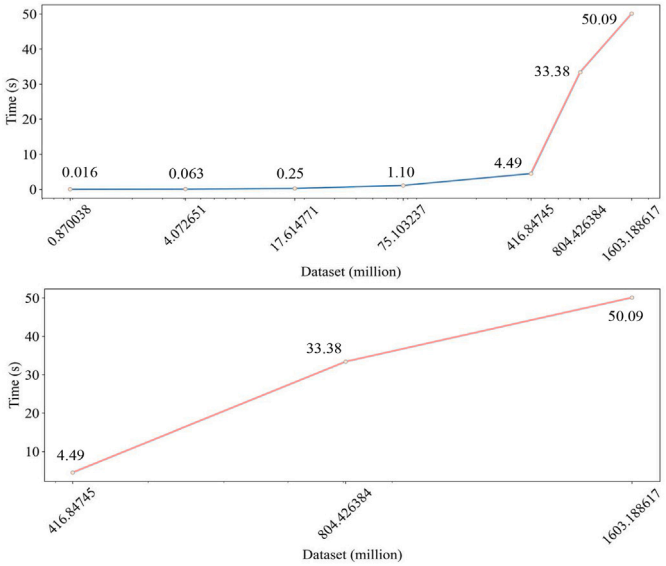


Fig. 17. The time complexity under different data scales.

4.3. Visibility query performance for large-scale datasets

Q-view combines indexing, spatial querying, and parallel computing to efficiently perform visibility analysis on large-scale datasets. To evaluate the impact of dataset scale on the algorithm's performance and runtime, we used datasets of various sizes and measured the average visibility query performance and runtime for each scale.

We conducted tests and analyses on datasets with meshes counts of 100,000, 1 million, 10 million, 100 million, and 1 billion, corresponding to Dataset-1 through Dataset-7. The dataset sizes grow approximately exponentially. Dataset-6 and Dataset-7 represent the oblique 3D models of Hong Kong Island and the Kowloon Peninsula, respectively. For each dataset, we set the number of threads to 16 and randomly selected 10 observation points, generating visibility results across the entire data range. We then calculated the average query performance and mean runtime. As shown in Table 5, the results indicate that the number of intersection operations between the LOS and the spatial index ranges from 2.2 million to 4 million per second. The overall analysis rate reaches millions of queries per second, demonstrating the algorithm's suitability for even larger datasets and its capability to support real-time visibility analysis on massive data scales

Next, we analyze the time complexity of Q-view on different dataset. Assuming the triangular mesh data scale is m, the time to traverse the spatial index structure is $O(log(m))$. When the viewshed radius is n, the time complexity is $O(n^2 log(m))$. As shown in Fig. 17, when the radius increases from 250 meters to 4000 m, the time grows by more than a factor of four, with fluctuations due to the variable k. As the dataset size grows to 800 million triangular meshes, the memory consumption of the spatial index becomes significant, impacting the efficiency of parallel computing and causing fluctuations in computation time.

Compared to traditional methods, Q-view does not require interpolation of terrain point elevations. Instead, it builds a spatial index for the triangular meshes and directly queries this spatial index during viewshed calculation, thereby significantly reducing the algorithm's complexity. Testing and analysis on massive datasets indicate that the visibility analysis performance of this algorithm gradually converges.

4.4. Parallel performance of visibility query

Visibility analysis between two points is data-independent, providing significant advantages in parallel computing. This section focuses on the speedup factor of parallel computing compared to serial computing. The number of threads was set to 1, 2, 4, 8, 16, 32, and 64. In Dataset-3, we randomly selected starting and target points for visibility calculations, performing 100,000, 500,000, and 1,000,000 parallel computations, respectively. We tested the parallel computing time with dynamic scheduling and the speedup ratio for different thread counts. Eq. (9) calculates the speedup ratio.

$$\eta = \frac{T_{p=1}}{T_{p=n}}, n = 1, 2, 4, 8, 16, 32, 64 \tag{9}$$

Here, $T_{p=1}$ and $T_{p=n}$ represent the execution times for serial and parallel computing, respectively. Fig. 18(a) illustrates that as the number of threads increases exponentially, the runtime for visibility analysis between two points decreases logarithmically. This reduction becomes more pronounced as the computational load increases. However, when the thread count reaches 32 or more, the optimization effect plateaus or may even slightly decrease. This is due to hardware limitations, as the machine supports a maximum of 32 threads. Exceeding this limit leads to frequent context switching and increased communication and synchronization overhead among threads, consuming extra computational resources and negating performance gains. Thus, simply increasing the thread count does not effectively reduce computation time.

Fig. 18(b) shows that the speedup achieved increases sequentially with the number of threads up to a certain threshold, reaching a maximum value of 16 for 100 million queries and a minimum value of 11 for 10 million queries with a thread count of 32. Within the range of physical core counts, the speedup ratio generally exhibits a nearly

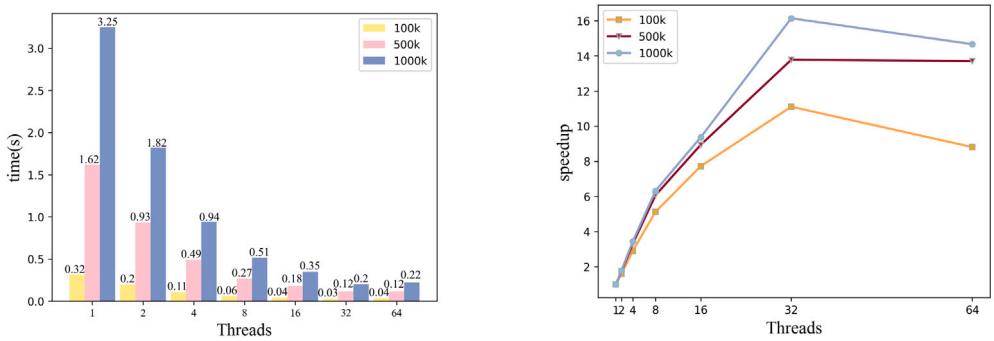


Fig. 18. Parallel experiment results. (a)Time consumption. (b)Speedup.

Table 6
Curve parameters.

	Start position	End position	Velocity (m/s)	Angle (degree)	Radius (m)
Curve-1	(114.166E,22.279 N,1 m)	(114.168E,22.282 N,1 m)	50	60	100
Curve-2	(114.166E,22.2825 N,1 m)	(114.166E,22.280 N,1 m)	50	50	100

linear growth trend, indicating that our algorithm can effectively leverage parallel computing advantages in executing visibility calculations, significantly enhancing the algorithm's performance.

4.5. Curve visibility analysis

Visibility analysis is generally suitable for visible light or simple electromagnetic environments. However, for applications like trajectory visibility for unmanned vehicles, curve visibility requires further investigation. One straightforward solution is to approximate the curve using a series of straight segments and then apply the Q-view algorithm for multi-point visibility analysis.

Using Dataset-9, the parameters of the curve are shown in Table 6. Assuming the object is only influenced by gravitational acceleration, its trajectory exhibits an ideal parabolic shape. The computation for curve visibility analysis is divided into two steps. First, the parabolic curve is approximated with straight segments through equidistant sampling. Starting from the launch point, we iteratively calculate whether each segment collides with any obstacles, preserving the sampling points that do not experience collisions. The second step generates a viewshed overlay matrix within the observation radius for each sampling point. This process achieves parabolic curve visibility analysis.

Figs. 19(a) and 19(c) illustrate the visibility of the curve between the start and target points. Green segments represent the trajectory unobstructed by obstacles, while red segments indicate obstructions by buildings. Figs. 19(b) and 19(d) show the viewshed overlay matrix of the unobstructed trajectory segments, with shades of green indicating the cumulative viewshed effect of multiple points. The Q-View effectively analyzes obstructions from complex building structures, enabling detailed 3D viewshed analysis applicable to real-time viewshed coverage for drones.

5. Conclusions and future work

In this paper, we propose a new paradigm for viewshed analysis that transforms traditional methods into a spatial viewshed query problem. The proposed Q-view is superior in accuracy and efficiency metrics. A series of experiments were designed to evaluate the performance of various algorithms: experiment 1 demonstrates that the Q-view is more accurate than R3, the most precise of the traditional methods. Theoretical accuracy falls short of the depth provided by Q-view; experiment 2 shows that Q-view outperforms the latest method by 72% in efficiency; experiment 3 explores the visibility query performance of Q-view on datasets of various scales, revealing that it can perform up to 4 million calculations per second; experiment 4 analyzes the speedup ratio under different threading conditions, affirming its compatibility with parallel computing frameworks; in experiment 5, the feasibility of curve visibility analysis is examined, enabling potential applications in trajectory occlusion, path planning, and virtual reality. These experiments present a promising solution for efficient viewshed analysis in large-scale and detailed scenarios.

Compared to other 2D and 3D GIS software, Q-view currently offers limited functionality and lacks a well-developed visualization interface. At present, it achieves visualization by projecting 3D spatial queries into 2D raster images and overlaying them onto the model. As a result, the display of observation points on surfaces such as rooftops, walls, and the ground has some limitations. In future work, we will focus on developing visualization techniques specifically for triangular meshes. In addition, we will focus on integrating advanced statistical analysis and semantic segmentation to support broader decision-making applications for large datasets. For example, we plan to optimize UAV route planning for aerial monitoring of urban transportation or natural disasters, enabling more intelligent and versatile solutions.

CRediT authorship contribution statement

Yifan Zhang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Mengyu Ma:** Resources, Funding acquisition, Formal analysis, Conceptualization. **Jun Li:** Project administration, Data curation, Conceptualization. **Anran Yang:** Validation, Software, Data curation. **Qingren Jia:** Writing – original draft, Funding acquisition, Conceptualization. **Zebang Liu:** Writing – original draft.

Software and data availability

Name of the code/library: Q-view: An Efficient and Fine-Grained Viewshed Analysis method in a Three-Dimensional Urban Complex Environment

Developer: Yifan Zhang, Mengyu Ma.

Contact: zhangyifan.000@nudt.edu.cn

First year available: 2024

Hardware requirements: PC

Program language: c++ and python

Software required: proj, CGAL, mpirun

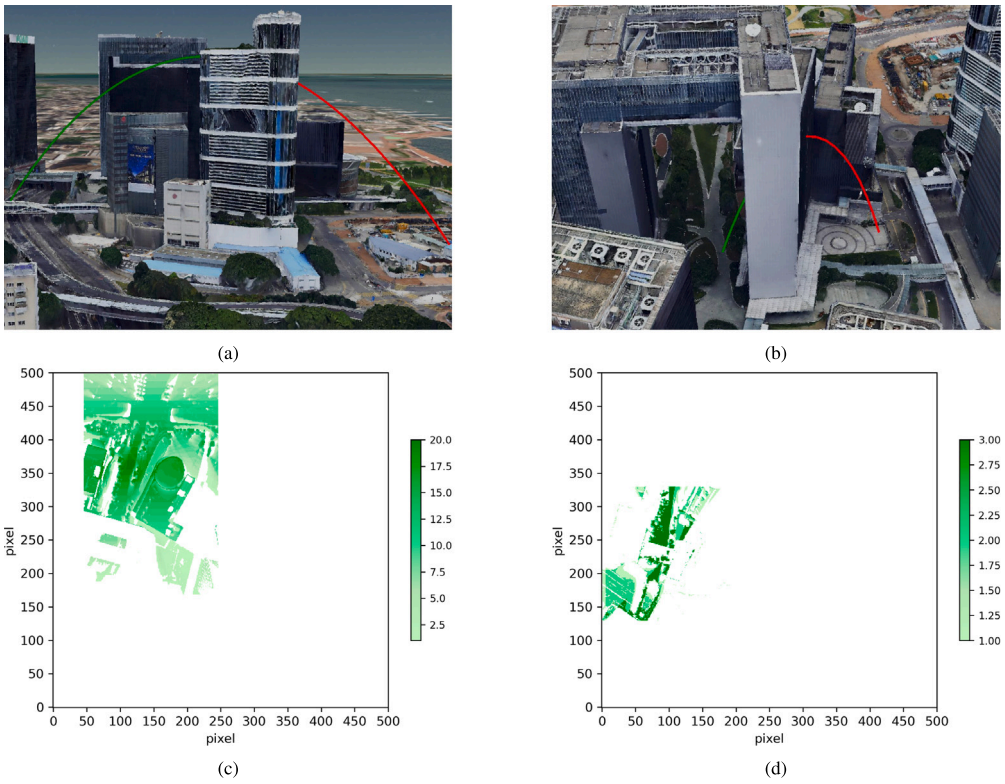


Fig. 19. Curve visibility visualization. (a-b) Curve visibility. (c-d) Viewshed overlay matrix.

Program size: 6.87M

The source codes are available for downloading at the link: Q-view

The datasets are available for downloading at the link: CSDI

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by National Natural Science Foundation of China (Grant number 42101432,42101435), the Hunan Provincial Natural Science Foundation (Grant number 2022JJ40546) and the Innovative Research Group Project of the National Natural Science Foundation of China(Grant number 42471403)

Data availability

https://github.com/Zoey-Fan/Q-view.

References

Anderson, C.C., Rex, A., 2019. Preserving the scenic views from North Carolina's Blue Ridge Parkway: A decision support system for strategic land conservation planning. Appl. Geogr. 104, 75–82.

Axell, T., Fridén, M., 2015. Comparison between GPU and parallel CPU optimizations in viewshed analysis.

Blelloch, G.E., 1990. Vector Models for Data-Parallel Computing, vol. 2. Citeseer.

Chen, Y., Chen, J., 2021. A parallel multipoint viewshed analysis method for urban 3D building scenes. Trans. GIS 25 (4), 2010–2028.

Dungan, K.A., White, D., Déderix, S., Mills, B.J., Safi, K., 2018. A total viewshed approach to local visibility in the Chaco World. Antiquity 92 (364), 905–921.

Feng, W., Gang, W., Deji, P., Yuan, L., Liuzhong, Y., Hongbo, W., 2015. A parallel algorithm for viewshed analysis in three-dimensional digital earth. Comput. Geosci. 75, 57–65.

Franklin, W.R., Ray, C.K., Mehta, S., 1994. Geometric algorithms for siting of air defense missile batteries. Res. Proj. Battle 2756.

Gaube, V., Remesch, A., 2013. Impact of urban planning on household's residential decisions: An agent-based simulation model for Vienna. Environ. Model. Softw. 45, 92–103.

Guttman, A., 1984. R-trees: A dynamic index structure for spatial searching. In: Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data. pp. 47–57.

Helmrich, A.M., Ruddell, B.L., Bessem, K., Chester, M.V., Chohan, N., Doerry, E., Eppinger, J., Garcia, M., Goodall, J.L., Lowry, C., Zahura, F.T., 2021. Opportunities for crowdsourcing in urban flood monitoring. Environ. Model. Softw. 143, 105124.

Jung, J., Olsen, M.J., Hurwitz, D.S., Kashani, A.G., Buker, K., 2018. 3D virtual intersection sight distance analysis using lidar data. Transp. Res. Part C: Emerg. Technol. 86, 563–579.

Labib, S., Huck, J.J., Lindley, S., 2021. Modelling and mapping eye-level greenness visibility exposure using multi-source data at high spatial resolutions. Sci. Total Environ. 755, 143050.

Li, W., Franklin, W.R., de Magalhães, S.V.G., Andrade, M.V., 2017. Gpu-accelerated multiple observer siting. Photogramm. Eng. Remote Sens. 83 (6), 439–446.

Ma, X., Zhang, X., Zhao, X., 2023. Service coverage optimization for facility location: considering line-of-sight coverage in continuous demand space. Int. J. Geogr. Inf. Sci. 37 (7), 1496–1519.

Ma, Y., Zheng, Y., Cheng, J., Zhang, Y., Han, W., 2019. A convolutional neural network method to improve efficiency and visualization in modeling driver's visual field on roads using MLS data. Transp. Res. Part C: Emerg. Technol. 106, 317–344.

Möller, T., Trumbore, B., 2005. Fast, minimum storage ray/triangle intersection. In: ACM SIGGRAPH 2005 Courses. 7–es.

Orlof, J., Ozimek, P., 2021. TIN surface and radial viewshed determination algorithm parallelisation on multiple computing machines. Symmetry 13 (3), 424.

Parent, J.R., Lei-Parent, Q., 2023. Rapid viewshed analyses: A case study with visibilities limited by trees and buildings. Appl. Geogr. 154, 102942.

Preparata, F.P., Shamos, M.I., 2012. Computational Geometry: An Introduction. Springer Science & Business Media.

Ray, C.K., 1994. Representing Visibility for Siting Problems. Rensselaer Polytechnic Institute.

Sabino, R., Vidal, C.A., Cavalcante-Neto, J.B., Maia, J.G.R., 2023. Building oriented bounding boxes by the intermediate use of ODOPs. Comput. Graph..

Sanchez-Fernandez, A.J., Romero, L.F., Bandera, G., Tabik, S., 2021. VPP: visibility-based path planning heuristic for monitoring large regions of complex terrain using a UAV onboard camera. IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 15, 944–955.

Saxena, V., Kaur, N., 2022. Comparison of R2 and R3 approaches over CPU vs. GPU implementation. In: 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom). IEEE, pp. 377–379.
Shapira, A., 1990. Visibility and terrain labelling. (Master's thesis). Rensselaer Polytechnic Institute, Troy, NY.
Shi, C., Yang, Q., Zhao, X., Shi, S., Yibulayimu, S., Liu, J., Wang, Y., Zhao, C., 2023. Fast and precise collision detection for detailed and complex physiological structures. Comput. Methods Programs Biomed. 240, 107707.
Stucky, J.L.D., 1998. On applying viewshed analysis for determining least-cost paths on digital elevation models. Int. J. Geogr. Inf. Sci. 12 (8), 891–905.
Wang, M., Cao, J., 2021. A review of collision detection for deformable objects. Comput. Animat. Virtual Worlds 32 (5), e1987.
Wang, Y., Dou, W., 2020. A fast candidate viewpoints filtering algorithm for multiple viewshed site planning. Int. J. Geogr. Inf. Sci. 34 (3), 448–463.
Wang, Z., Xiong, L., Guo, Z., Zhang, W., 2023. A view-tree method to compute viewsheds from digital elevation models. Int. J. Geogr. Inf. Sci. 37 (1), 68–87.
Wróżyński, R., Pyszny, K., Wróżyńska, M., 2024. Reaching beyond GIS for comprehensive 3D visibility analysis. Landsc. Urban Plan. 247, 105074.
Wu, C., Guan, L., Xia, Q., Chen, G., Shen, B., 2021. PDERL: an accurate and fast algorithm with a novel perspective on solving the old viewshed analysis problem. Earth Sci. Informatics 14 (2), 619–632.
Yao, Y., Feng, C., Xie, J., Yan, X., Guan, Q., Han, J., Zhang, J., Ren, S., Liang, Y., Luo, P., 2023. A site selection framework for urban power substation at micro-scale using spatial optimization strategy and geospatial big data. Trans. GIS 27 (6), 1662–1679.
Zhang, G.-t., 2021. Visual landscape analysis for urban mountains based on multi-dimensional digital models. (Ph.D. thesis). Southeast University.
Zhao, Y., Padmanabhan, A., Wang, S., 2013. A parallel computing approach to viewshed analysis of large terrain data using graphics processing units. Int. J. Geogr. Inf. Sci. 27 (2), 363–384.